Development Plan
Software Engineering

Team 18, Team Magic
Allen Tong
Amos Edeaghe
Jacob Brodersen
Joshua Engel
Nehan Munir

Table 1: Revision History

Date	**Developer(s)**	**Change**
September 27, 2026	Nehan Munir	Added IP, Confidential Information and Communication plan
September 27, 2026	Josh Engel	Added Proof of concept
September 27, 2026	Nehan Munir	Added Team communication plan and Team member roles
September 27, 2026	Nehan Munir	Added Work Flow Plan and Project Decomposition
September 28, 2026	Josh Engel	Added Team charter
September 28, 2026	Amos Edeaghe	Added Expected Technology and Coding Standard
September 28, 2026	Allen Tong	Added Copyright License and Use of AI
September 28, 2026	All members	Added individual reflections and chose roles

The purpose of this document is to outline the development plan for Magic 8 found here, a solitaire game focused on finding missing numbers on an electronic 8x8x8 magic cube. This will contain information on how the team is set up and will meet, the workflow they will be using, alongside their PoC Demonstration Plan and their AI usage.

1 Confidential Information?

There is no confidential information to protect as this project will be based on existing, open access literature.

2 IP to Protect

There is no IP to protect, and the team has agreed to let Dr. Paul Rapoport post it for free on a website of his choosing.

3 Copyright License

This project is licensed under the MIT License. The full license can be found at magic-8/LICENSE.

4 Team Meeting Plan

4.1 Team Meetings

The team will meet weekly to discuss project progress, upcomming tasks, and any issues requiring discussion. Weekly meetings will be in person at Thode group study rooms unless certain conditions require an online meeting. This meeting will occur on Wednesday 3:30 to 4:30 PM unless otherwise stated. Further meetings can be arranged in case of time sensitive decisions or emergencies.

4.2 Supervisor Meetings

The team will meet with the supervisor whenever they require the supervisors input on an issue or task. These meetings will take the place of the weekly team meeting unless the supervisor requires another time. There is also a teams chat with Dr. Rapoport for quick communication outside of scheduled meetings when he is available.

4.3 Meeting Structure

Each meeting will have a dedicated chair responsible for ensuring the agenda is met within the time limit of the meeting. An agenda must also be prepared before scheduled meetings to allow for efficient and useful feedback on problems

the team may have throughout the project. At the end of each meeting there will be a review of current and future work to ensure work is being split fairly and efficiently. Meeting notes will then be shared to the team to ensure anything that happened in the meeting is properly documented.

5 Team Communication Plan

The team will be using the following communication tools and methods to ensure that this is a successful project.

- **Github**: There is a Github repository that will contain the code and documentation for this project. There will be two releases: one that will allow you to run the project as an application, and another which will be used for Dr. Rapoport's prescribed website. In lieu of assigning tasks in passing and promptly forgetting, there will be a Github Projects board that will follow the Kanban method taught in 3RA3, with tasks assigned simultaneously during meetings. Meeting minutes will also be uploaded to the repository.
- **MS Teams**: There is a Teams chat with all 5 group members. This will be the standard place for communication outside of in-person meetings and can be used as either a place for virtual meetings or as an asynchronous chat. Ideation outside of meetings will occur on an appropiate shared Words document, which will be committed onto Github when appropiate. There is also a seperate shared group chat with Dr. Rapoport, which will be used to schedule meetings and ask quick questions. This can also be used as a place to meet virtually with him, but in-person meetings are preferred.
- **Emails/Google Meet**: When corresponding with outside channels, such as other professors and/or stakeholders, emails and Google Meet are the preferred methods. When emailing, the message should be approved in the Teams chat before being sent, and it should be carbon-copied (CC) to all other members and Dr. Rapoport. Google Meet was chosen as stakeholders are lekely tech illiterate and Meet is an accessible browser-based meeting platform.

6 Team Member Roles

While the team has different strengths, there will be a lot of learning on the fly done for this project, so the team didn't feel comfortable assigning roles yet. This will be done after further discussion and after everyone has taken a stab at deeply exploring the subject matters at hand (ex. 3D visualization). The load and therefore role might end up falling on multiple shoulders or changed, if we see that it is improbable to be completed by one person. The document will then be revised appropiatedly.

Jacob Broderson will serve as the **Team Liason**, as he has already kept up communication with the supervisor via emails before moving to the Teams chat, and will also be responsible for communicating directly with the course's professors and teaching assistants should the need arise. He will also be primarily responsible for seeking out stakeholders and scheduling meetings with them.

Here are the following unassigned roles, based off of our approved project description:

- **Systems Analyst - Nehan**: As this project deals with a stringent set of rules that makes the Magic Cube work in octal, this person will be responsible for first compiling and creating all of the rules in our chosen programming language. He is reponsible for ensuring that none of the other members' work break these rules, using test cases and static/dynamic analysis tools. As the design space for octal magic cubes is unfinished and there are only 384 confirmed cubes, it is possible for this member to not only discover new ones, but also make/edit the base rules used to make cubes.

- **Game/Creative Director - Jacob**: Basing their work on the rules set by the Systems Analyst and the model/UI produced by the Front End Developer, the Game/Creative Director will focus on rules for the levels of the game, making sure that they are engaging while still educational. Dr. Rapoport already has an idea for the game in mind, and the Game/Creative Director will work in lockstep with him to find a middleground that is perfect with the stakeholders. This director will work in implementing and testing stakeholder feedback, and will also be responsible for thinking up and integrating new game modes.

- **Front End Developer (and 3D Artist) - Joshua**: They will be responsible for creating the user interface based on what the Game/Creative Director requires. They will also learn about 3D modelling/visualization on the side and create the needed visualization that can be efficiently displayed on a website.

- **Back End Developer - Amos**: This role requires the group member to store the required data (such as the 384 possible cube formations) in an accessible database, and create the pipeline and interfaces that makes it easier for the other developers to access the information. They can also be responsible for account management (especially important for keeping track of user scores and if the team implement an online mode) and other managerial interfaces that simplify the jobs for other developers. If the team end up choosing to do a family of games, they will need to come up with a creative way to support multiple databases of different sizes.

- **Webmaster - Allen**: Based on what Dr. Rapoport wants to do display the information (either as an executable linked on a website, or as a website itself), the Webmaster is responsible for writing and deploying the code that displays the work of the Front and Backend Developer. They will

have to ensure that the website is accessible to stakeholders in high school, which means using a public domain instead of using localhost or a private server.

7 Workflow Plan

Git will be used to keep track of progress, new features that have been brainstormed, and as a stable workflow where we can always point to what everyone is working on and our latest stable project.

After being assigned a task/issue from the Github Projects Kanban Board, the member will create their own branch from a corresponding primary branch (not main). The main branch will always be stable, and almost have same status as a new release (it shouldn't be merged into without being peer reviewed and without the express approval of every member).

When the team starts coding and building documentation after being assigned a role, they will start by creating a primary branch named after their role. For example, the Systems Analyst would call their primary branch, systems-analyst. Whenever there are tasks that requires multiple team members, they create a temporary primary branch named after the members using the branch. The goal of the users on this branch is to merge this branch into main as soon as possible as the mental calculation is that any feature requiring multiple members is worthy of being a release.

The team member will be allowed to experiment on their primary branch, as they can essentially create sub branches via "git branch role/experiment-description role" (ex. git branch systemanalyst/experiment-adding_new_rules systemananalyst). The member's primary branch should also be stable and when they get assigned an issue, they will move the issue from the "To Do" column to the "In Progress" column. After this has occurred, the person will work on a subbranch they made on their primary branch. For example, the Back End Developer would use the following command: "git branch backenddeveloper/feature-adding_magic_squares backenddeveloper". This will create a copy of the latest commits on backenddeveloper into the subbranch.

When they are finished working, they will have to git checkout into the primary branch and then create a merge request between the two branches (ex. going to backenddeveloper and making a merge request between backenddeveloper backenddeveloper/feature-adding_magic_squares). They will then move the issue to the "In review" column and add the merge request into the task on the Github Projects Kanban Board. They will also remove the previous tag (ex. remove "feature" or "refactor"), add "review", and assign another team member to the issue. These requects will only require a single peer review before being moved to "Done" and being accepted.

There can also be a primary branch for documentation, however these aren't as strict as actual code requests and the only requirement to merge into main is that the primary branch (ex. devplan for this document) is stable.

In terms of issues outside of the coding, they will also be used to track

meetings with the group, professors (during lectures) and our supervisor. There will be a title and a checklist for atendance. Outside of this, it will be a place for people to store their questions that can be updated, so everyone is aware of the questions that will be asked and to prevent forgetfullness.

This is more important for documentation as Github Issues is an easy way to assign parts of the written work to team members. There will be a main issue named the deliverable (ex, Development Plan) with a hard deadline. There will then be titled subissues with a soft deadline and an assigned member (ex. Workflow Plan), and they will include a self-contained checklist with short descriptions (ex. "How will you be managing issues, including template issues, issue classification, etc.?").

The CI pipeline would consist of pylint and strict formal requirements (written by the Systems Analyst) in pytest. This was chosen because it also checks for style issues (compared to pychecker, however the team will need to test their imports beforehand). There will also be static tests that were created by the Systems Analyst consistently running on the code when it has been commited alongside domain-specific tests written by the specific "branch manager". Based on what the team member is writing, there might be specific configurations that the member might use (such as the Front End Developer requiring a plugin to display their model). This means that the JSON/YAML file will also have to build before it can it can be merged. There is also a code review in order to ensure that small problems don't become big problems when it comes time to merge from the primary branches into main.

There is also a CD component if the team ends up publishing the website earlier to get user data, which will require an appropiate rollback strategy. This will ensure that data leaks (such as user data) are mitigated as soon as possible.

8 Project Decomposition and Scheduling

The Github project is located at: Magic 8 Kanban

The Kanban board has the following columns: Lectures/General Ideas, Meetings, To Do, In Progress, In Review, Done.

The Lecture column will contain labels that describe it as either "Attendance" (to ensure that everyone is able to get the required knowledge to complete milestones and its corresponding documents) and/or "Questions" (if the team has questions in the descriptions that they want to ask the professor after class). There is also space to create issues that are just spitballed ideas.

Meetings will be labeled to show who it is with, and will be populated with pre-meeting questions. During the meeting, meeting minutes will be added to the desciption, alongside roles/tasks assigned. After the meeting is concluded, it will be labelled as "Not Assigned", until the task discussed in meetings are turned into issues (in "To Do") that are labeled to members. The rest of the board will follow the guidelines set in Workflow Plan, alongside belonging to the appropiate milestone listed below:

Milestones	Week
Team Formed, Project Selected	Friday, September 18
Problem Statement, POC Plan, Development Plan	Monday, September 28
Req. Doc. and Hazard Analysis Revision 0	Monday, October 19
V&V Plan Revision 0	Monday, November 2
Design Document Rev -1 and User-Stakeholder/ Client-Interview Report Rev 0	Monday, November 16
Proof of Concept Demo + Interviews	Monday, November 23 + 30
Design Document Revision 0	Monday, January 18
Revision 0 Demo + Interviews	Monday, February 1 + 8
V&V Report Rev 0 and Usability Report Rev 0	Monday, March 8
Final Demonstration (Revision 1)	Wednesday, March 31
EXPO Demonstration	Tuesday, April 6
Final Documentation (Revision 1)	Tuesday, April 6

The Final Documentation will contain the Problem Statement, Development Plan, Proof of Concept (POC) Plan, Requirements Document, Hazard Analysis, Design Document, V&V Plan, V&V Report, CD-Fall, CD-Winter and the Source Code.

9 Proof of Concept Demonstration Plan

Two of the greatest risks currently identified for this project are the 3D visualization of the magic cube and the implementation of the base design of the game and its eight stages. The 3D visualization represents the largest risk that the POC will attempt to address. This is because, for the game to function as currently envisioned, 3D visualization is a hard requirement; however, the group has little to no experience working with 3D visualization. The other risk to be addressed is the base logic of the game itself. This has been identified as a risk because if the base functionality cannot be implemented, then any further development would be ineffective.

To address these risks, the POC will consist of a minimal version of the game in which we will be able to demonstrate a user moving through a 3D magic cube and completing the eight tasks outlined in the project description. These tasks will be simplified and hardcoded versions of the final product, where the numbers provided to the user will always remain the same. This will allow us to test the functionality of the tasks without requiring the random elements of the game to be implemented. The POC will be considered successful if the magic cube can be rendered and interacted with in 3D, and if the user can successfully complete all eight tasks without any critical issues with the underlying game logic.

If the demo were to fail, the consequences for the project as a whole would depend on which risk caused the failure. If the implementation of the eight stages of the game were to fail, a significant discussion regarding the future of the project would be necessary. This would indicate that the group is unable to deliver the fundamental functionality required by the project and may require a significant deviation from the original capstone plan. If the 3D visualization were to fail, the group would instead need to evaluate the possibility of transitioning to a static or 2D version of the visuals rather than continuing with the current 3D plan.

10 Expected Technology

Due to familiarity the team will likely use Python as the main language to build the project with consideration for JavaScript as for the web frontend. For current consideration on game implementation the team expects to use the following libraries: the pygame community edition pygame-ce library for implementing the game logic and functional loop, a 3D engine such as Panda3D or Harfang3D, if Panda3D provides issues, for asset handling and 3D rendering, pygbag to convert the application into WebAssembly packages to allow the game to run efficiently as a webgame, and asyncio for asynchronous event handling to run logic on the browser between game frames with further libraries to be discussed as the project develops such as the possible use of NumPy or researching JavaScript libraries. The team also plans to make use of readily available assets for the game with agent design for the cube visuals or generating new cubes in different bases when pursuing our stretch goals that would make use of AI agents such as Claude.

For tools team members will use their preferred IDE with the most commonly used being Visual Studio Code, GitHub, GitHub projects and Git will be used for version control as laid out in the Workflow Plan. As continuous integration is planned we will make use GitHub Actions to run our testing, build and linting tools for each push, pylint will be used as our linter to match our chosen coding standard, pytest and pytest-cov will be used for running unit tests and checking code coverage, mypy is being considered for static type checking.

For performance measurement the team is considering the use of cProfile to find points for optimization by measuring the execution time for the different functions such as rendering or task updates, timeit would be used in conjunction to compare the speeds of different implementations, and the tracemalloc library would be used for memory tracing to find leaking or unnecessarily large memory blocks. All listed libraries and implementations are subject to change based on the team's needs and exploration of better alternatives or as changes are made for aspects the team would later prefer to implement ourselves such as the cube visualization or algorithm analysis.

11 Use of AI and LLMs

The team expects to use AI tools, such as GitHub Copilot and Claude, at various stages of the project during both development and documentation. Most uses will fall under brainstorming, creating boilerplate code, and assisting with debugging.

Since the game's core logic is based on a fixed, formula-driven set of rules, any AI-generated implementation of these rules will not be merged until it has been checked against the correct magic sum (2052 decimal / 4004 octal) using the static test suite described in the Workflow Plan. AI-generated 3D visualization code will similarly be manually tested, since correctness in this area cannot be verified by unit tests alone.

For documentation, AI tools may be used to help draft, restructure, or brainstorm sections of project documents. Any AI-assisted content will be reviewed by the team for accuracy before inclusion, particularly where it touches the mathematical rules of the game or project requirements.

Regardless of the source, all code passes through the same merge request and peer review process described in the Workflow Plan. AI-generated contributions are held to the same standard as human-written ones and do not skip any step of the process.

12 Coding Standard

As the team expects to use Python the project will follow the Google Python Style Guide for formatting for consistency in naming, documentation, type annotation and coding practices. The standard will be enforced through pylint static checking and Google's recommended Vim files or autoformatters.

13 Appendix — Generative AI Use Disclosure

13.1 AI Tools Used

Claude Sonnet 5 (Medium) by Anthropic M365 Copilot by Microsoft

13.2 How and Where Used

Claude was used to brainstorm ideas for how the team can effectively use AI and LLMs throughout the project. These ideas were used to help write the Use of AI and LLMs section. Copilot was used for recommendations on libraries to use based of teams requirements and initially planned libraries. These suggestions were considered for the writing of the Expected technology section along with its use in spelling error checks for the same section.

13.3 Verification of AI-Generated Content

The team reviewed and edited the AI-generated ideas before including them, and verified that they were consistent with the workflow, testing, and peer review processed described in the Workflow Plan. The team remains responsible for the accuracy and completeness of all content in this document, regardless of whether it was generated by AI or written by a team member.

Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. Why is it important to create a development plan prior to starting the project?

2. In your opinion, what are the advantages and disadvantages of using CI/CD?

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Allen Tong

1. A development plan is important to create before starting the project because it gets the team to agree on how we will work before any code is written. Deciding on how we will communicate, manage our code, and track tasks is important to ensure that work is not assigned in passing and forgotten, which helps the project run smoothly. It also made us identify our biggest risks, which are the 3D visualization and the base logic of the game, allowing us to plan our Proof of Concept around them to avoid delays later on. Finally, it gives us a shared reference to check against if someone falls behind or the team disagrees on the direction of the project.

2. The main advantage of using CI/CD is that every merge request is checked automatically before it is merged into the main branch. This is especially important for our project because the game rules are exact, so a test confirming that every line sums to the correct magic constant will catch any mistakes right away. A disadvantage is that it can be time-consuming to set up and maintain the pipeline, especially since most of us are still learning the tools involved.

3. Our group did not have any disagreements during this deliverable. We split up the tasks beforehand so that everyone knew what to do and what was expected of them. As a result, the team worked well together and everyone understood what the document needed.

Josh Engel

1. I believe it is very important to create a development plan prior to starting a project. By doing this you are minimizing risk of issues such as scope creep since everyone is in agreeance with the initial scope of the project. It also allows for a consensus to be reached between team members on several important factors to the project such as tools used and team expectation. By reaching such a consensus you minimize the risk of large arguments within the group during more time crucial moments of the project.

2. In my opinion CI/CD is a very advantageous tool due to it allowing for much easier and consistent testing of the codebase of the project. But this does come with a large overhead of the initial setup of CI/CD which can tend to be complicated especially for those with little experience such as us.

3. I would not say that we had any major disagreements within this deliverable. Differences of opinion were brought up but they were all managed gracefully with options like a vote to resolve them.

Jacob Brodersen

1. It is important to create a development plan prior to starting the project for a few different reasons.

   First, the development plan ensures that everyone is on the same page. For example, it is important that everyone agrees with and understands the team communication plan so that we can achieve effective and efficient communication.

   Second, the development plan holds team members accountable. This document can act as an informal contract of the standards expected from each group member.

   Third, the development plan reduces risk. By planning ahead, inconsistencies and potential roadblocks can be identified ahead of time. It is much less costly to fix/mitigate a risk now than it would be in the later stages of the project.

2. One advantage of using CI/CD is that it forces all merged code to have specific standards. Therefore, mistakes in these standards can be caught early. This can also improve collaboration as everyone is held to the same CI/CD specifications.

   A second advantage of using CI/CD is that it is an automated process. This takes some of the manual workload away and eliminates the risk of human error (except when making the CI/CD itself).

   One disadvantage of using CI/CD is the initial time required to set up. Although, this can be reduced using templates / premade systems as we

do in this project. Additionally, CI/CD can be quite confusing for a new user or even for a new CI/CD layout.

3. There were initially some disagreements revolving around the Expected Technology section. At first, each team member had different ideas of what technologies (e.g., programming languages and libraries) they wanted to use. To resolve this, we met live and allowed each person to explain the pros and cons of their proposed technologies. At the end, we used a majority voting system to decide on the final technologies to use.

Nehan Munir

1. The purpose of a development plan is to ensure the team is on the correct page in terms of what they want to accomplish during this project. For example, outside of the team charter and meeting, the expected technology section was written after agreeing on what languages we will be using for the rest of the project. This might mean that we have to learn languages on our own, but now we have written acknowledgement of this fact. Another example is the team roles and workflow plan, setting expectations for the next eight months.

2. One of the advantages of CI is that documentation is automatically builds from our tex files. This means that we don't have to manually build every time we commit our work, saving time. The CI/CD pipeline we have is already descriptive, so we are aware of any latex errors and exactly where it stops. We are going to have to build a pipeline for automatically checking if merge requests passes all of our tests, given that we have strict rules that the main branch and primary branches have to be stable. The Systems Analyst will have to ensure that they all plass the strict formal specifications that the magic cube has. CD might also be important given that we are running a website, which will likely be deployed m,ultiple times just to gather data points and feedback from users. A disadvantage to not building your own pipelne is that it is hard to fix when you know what you are writing is stable, but the pipelne is showing errors. We are having build problems for our first submission, which I believe is due to not having a prepopulated cache with the correct libraries. This was a .yml file that we had to fix.

3. There was too many hesitation when deciding on ideas and other stuff that might require a snap decision. This is very likely due to being a group that has never worked together before and perhaps due to our inexperience in creating the type of project we want to (for example, there will be 3D modelling [no one in our group has done that before], making a website [only one member], and creating a game [no members]). This problem came to a head when we were choosing team roles, and no one knew what role they wanted to do. If members were able to immediately choose a role, it would mean that they have anm eagerness to build the expected

part, which might mean they will go above and beyond to make something they are proud of. This conflict was resolved via having people carefully read the roles, and a team member making sacrifices for the betterment of the project.

Edeaghe Amos

1. I believe the development plan has helped me put into perspective the proper scope of the project after combining it with the problem statement. It makes it easier or team members to set their expectations and agree on the roles, role flexibility early on which I find are typically the hardest parts of team norming, It feels like through the completion of this all members have some idea on where they stand for the project and an idea of how it will progress.

2. With multiple members working from different locations and devices a CI/CD plan allows us to push our changes and be assured that any problems we didn't catch won't break the system it adds a safety net and provides a consistent standard we can all work with to further lower our errors, it also means we don't have to waste as much time with error handling speeding up our development and the continuous deployment aspect means we can continuously provide value in our implementations and lower our risk of stakeholder dissatisfaction as each delivery can get their feedback. As for disadvantages I would say creating a good pipeline takes a lot of effort for the tests, tools etc. and we can also fool ourselves with a faulty pipeline and miss major issues.

3. I did not notice a major disagreement this deliverable a communication and work split was handled early on.

Appendix — Team Charter

External Goals

The team's main goals are to create a well made and presentable project that can be used for reference dureing job interviews in the future. All team members should be proud of the final result that is achieved and see it as a good stepping stone in their journey. The team also hopes to provide a project worthy of a high grade in this course.

Attendance

All team members are expected to be in attendance at meetings with the supervisor Dr.Rapoport, while during intergroup meetings only those who have relevant knowledge and responsibilities to the agenda must attend. For all meetings including online meetings the attending members are expected to stay until the end unless they have an acceptable excuse, all team members should also arrive within five minutes of the agreed upon meeting time.

Expectations

All attending team members are expected to have prepared by reviewing any relevant documentation, issues, pull requests, or other materials. Work brought to these meetings is also expected to be sufficiently complete to allow for feedback in the meeting, and each member is expected to take responsibility for assigned work. This includes communicating progress as well as issues that they have faced attempting to implement it.

Acceptable Excuse

A list of acceptable excuses:

1. Academic commitments
2. Significant medical circumstances
3. Emergency requiring immediate attention

In Case of Emergency

If a team member is unable to attend a meeting they should notify all team members whenever possible. They should then follow up on what they missed with other team members. Otherwise if a team members expects to miss a deadline they should notify the team as well as the instructor to explain why.

Accountability and Teamwork

All team members should contribute evenly and remain accountable for their work. They should also communicate concerns or delays early to catch them before they cause an issue. Team members are also expected to contribute to a positive enviornment where collaboration is supported.

Quality

The team is commited to a standard of high quality work throughout the project. This includes the project itself which should be peer reviewable by other team members. As well as high quality communication allowing for efficient feedback on work as well as interpersonal issues.

Attitude

Team members should maintain a positive and professional attitude throughout the project. They shold have open minds to others ideas and be willing to consider alternatives. This also means treating other team members with respect including, giving constructive critisism, and focus on disagreements on the project instead of as individuals.

Stay on Track

Project progress will be tracked on the github kanban board. Team members are responsible for keeping their tasks up to date and communicating issues that occur. To help motivate group members, repeated failure to meet expectations may be escalated to a TA or instructor if deemed necessart by the group

Team Building

To help upkeep a postive attitude within the team throughout the project the team members are encouraged to have informal meetings where team building activities will occur. As well as having casual conversation after scheduled meetings, and celebrating large milestones in the project as a team.

Decision Making

Decisions will generally be made with a majority vote. Since the group consists of five members this will never result in a draw. If a major disagreement occurs and the vote is split 3-2 then the group will seek supervisor input and follow that to hopefully lead to a more agreeable solution.